Project Journey Portfolio

StudyOS Thesis-Finder — working draft

Group submission • Domenique, Max, Valentin • University of Tübingen

Info-gathering version. Each section mirrors the assignment structure (What / Evidence / So what / Now what) and states what we already have from the repository. Everything marked in an **Open** box is a question the team still needs to answer — these are the gaps between what is documented and what the assignment explicitly asks for.

The journey logic in one paragraph (draft): We started with a matching platform (app + database) for thesis search. Early conversations with professors broke the core premise: the supply side won't feed a central platform, and a curated database rots. We reframed the problem from "matching" to the student's cold start, pivoted to a database-free agent skill that searches the live web, built and self-evaluated it against a plain-Claude baseline, and shipped it as a public, installable package. What's still missing is testing in the hands of real students — the protocol is ready, the test hasn't run yet.

1 Initial Project Idea

Assignment asks for: initial idea, motivation, initial assumptions, first problem idea, target group, an early proposal/sketch/user story — plus why it seemed promising and what needed checking next.

We solve X for Y by doing Z

We solve **topic discovery** for **thesis students** by **bringing open theses into one place**

Our initial idea was a full-stack web app for students and professors. The professors could easily list their recent thesis topics while the students can search over the list of available topics and find their most suitable match. The motivation came from out own experience of searching and finding a suitable thesis topic. At the university there are multiple different data sources like the official university website, alma and additionally most of the professors also have a private lab based website. Our first hypothesis was, that the major problem is that there is no single source of truth. Therefore we have planned and developed a first full-stack prototype, with a backend, a database and a web frontend. Since we are in the age of AI, we also planned to integrate a full custom AI agent with custom tool calls, that could supervise students during the search process and the exploration of their own possibilities and goals for their thesis. There we fell into the trap that we as software engineers love to build complex systems, just because we can, not because it is actually needed.
Our initial target group were students and professors of the Department of Computer Science at the University of Tuebingen. We treated the students as the demand side and the professors as the supply side of our platform. After discussing the initial idea in the course a lot of students thought that the idea was promising, because a lot of them faced the same problem in their bachelor studies. Common stories were that students were frustrated in the end, because they didn't know about specific chairs, which might have been the perfect fit for their interests and skills. Another main point was that a lot of students reached out to professors, but didn't get a response. At this point we hadn't done the user research for the supply side yet, so we only had the students perspective and the perspective of Professor Gehler.

To mitigate this discrepancy, we planned a user studys for professors, where they could give us their point of view, without rating the actual idea itself. We wanted to understand the reasons

behind the missing thesis list and if it is intended or just fell of due to the lack of time.

While the whole application got stripped apart during the process of the course, the initial idea was a good starting point and a problem worth solving.

2 Discovering and Understanding Users

Assignment asks for: user groups discovered and selected, how they were reached, the research approach, main insights, a persona or user story — plus what we learned that the initial idea alone couldn't tell us, and how that shaped the direction.

What we have

- Two user groups, and this distinction matters for the story: students are the demand side, professors/chairs are the supply side a platform would depend on.
- We interviewed a substantial number of professors. Main documented insights:
  - Roughly half rejected a central platform outright; only about a third were conditionally open to it.
  - A comparable topic-listing platform at the university had already failed — for incentive reasons, not UI reasons: nobody wants to list and maintain open topics.
  - Individual professors reacted much more positively to a different framing than to our original platform idea.
- The course professor later gave feedback that shifted what the product is even *for*: the real value is that students reflect on what they want — not the concrete proposals the tool outputs. That insight is now built into the skill (it records the student's self-understanding before and after the search).

Evidence to attach

- Interview findings and quotes: `docs/thesis-report/00-problem-and-research/README.md` and the professor research package in the same folder.
- Inventory of every documented user contact, with quotes: `findings/no_db_universal_skill/2026-08-09-user-study-inventory.md`

So what (draft)

- The initial idea could not have told us that the *supply side* is the bottleneck. Interviews revealed the platform premise fails regardless of how good the product is — participation and freshness are incentive problems, not engineering problems.

Now what (draft)

- This forced the reframing in Section 3: solve the student's problem without depending on professor participation or on curated data.

Open — needs the team

- How were the interviews actually done — how were professors selected and reached, was there a fixed question guide or informal conversations, and who conducted them? The repo itself flags that this method was never written down.
- What student-side input existed before the pivot? Right now the documented pivot evidence is professor-side only, but the product targets students — if we talked to fellow students informally (friends, Fachschaft, ourselves as users), that must be described; if we didn't, we should say so honestly and explain why professor evidence was enough.
- Can we write one short persona or user story for our target student? The assignment asks for one explicitly and we have none.
- What is the single most surprising thing a user said to us? One strong quote carries more than a paragraph of summary.

3 Defining the Problem

Assignment asks for: initial vs. refined problem definition, which assumptions changed and why, and the fit between users, context, and problem.

What we have

- Initial problem definition (implicit): "students can't find matching advisors → build a matching platform."
- Refined problem definition: the hardest part is the **cold start** — thesis information is scattered across chair websites, students don't know what they don't know, and any curated database only covers one faculty and only stays correct while someone maintains it.
- The two questions that triggered the rethink are literally preserved in our meeting notes: *"Do we even need a backend?"* and *"Where's the difference to just using Claude without a skill?"*
- Assumptions that broke, with the evidence that broke them: professor participation (interviews), database freshness (the failed predecessor platform + our own maintenance backlog), and willingness to use a separate external tool (nobody opens yet another app for a one-time task).

Evidence to attach

- Raw pivot meeting notes: `docs/thesis-report/01-the-pivot/2026-06-25-besprechung-notes.md`
- Reframed problem statement: `docs/thesis-report/01-the-pivot/2026-06-26-vision-no-db.md`
- Decision log entry for the pivot: `docs/thesis-report/decision-log.md`

So what (draft)

- The refined definition is better because it survives the broken assumptions: it doesn't require anyone's participation, doesn't rot, and meets students where they already are. It also honestly shrinks the claim — from "we match you" to "we map your options."

Now what (draft)

- The refined problem directly dictates the solution constraints in Section 4: no database, no backend, live sources only, all faculties, zero maintenance.

Open — needs the team

- Write both problem definitions as one clean sentence each ("We aim to solve X for Y by doing Z"), initial and refined, so the before/after is visible at a glance. Draft them, then agree as a group.
- How did the team actually converge on the reframing — immediate agreement or real back-and-forth? One honest sentence about the group dynamic belongs here.

4 Proposing a Solution

Assignment asks for: solution concept, value proposition, alternatives considered, feature prioritization, and the fit between users, problem, and solution.

What we have

- Solution concept: a portable, database-free **agent skill package** — markdown instructions that teach an LLM how to interview a student about interests/skills/constraints and then search the live web in two passes (discover candidates from multiple source axes, then verify and enrich each one), producing a **map of options** with rationale and evidence, not a single answer.
- Value proposition (draft, needs a group-agreed sentence): always current because it reads the live web, covers all faculties immediately, requires zero maintenance, and lives inside the tool

students already use — no separate app to install or account to create.
- Alternatives considered, documented: the original app with database (built, abandoned); a database-backed skill vs. the database-free skill — genuinely contested inside the team, resolved by pursuing both in parallel until testing showed no-DB wins on both user value and maintenance cost.
- Why it beats "just ask Claude": the skill carries encoded discovery rules (source axes, verification checks, interview discipline) — this was reasoned out explicitly and later measured (Section 6).
- Feature prioritization, documented: university arm first and proven, companies as a second phase on the same principle, other universities and Bachelor's theses explicitly out of scope.
- The concept was deliberately stress-tested against named risks before we committed to it.

Evidence to attach

- Full solution vision: `docs/thesis-report/01-the-pivot/2026-06-26-vision-no-db.md`
- Risk grilling with decisions: `findings/no_db_universal_skill/2026-06-26-concept-and-risks.md`
- Scope and ordering rationale: `MASTERPLAN.md` §1–4

So what (draft)

- Chosen because it is the only direction that fits every constraint the refined problem imposed — everything database-shaped fails on maintenance, everything app-shaped fails on adoption.

Now what (draft)

- The proposal directly produced the build plan: prove the university arm against a ground truth and a plain-Claude baseline before extending anywhere else.

Open — needs the team

- Agree on the one-sentence value proposition — the assignment asks for it verbatim and the checklist checks for it.
- Were any alternatives seriously discussed beyond app-with-DB vs. skill-without-DB (e.g. keeping the app but dropping the DB, a browser extension, plain prompt templates)? If yes, name them and why they lost; if no, say the decision space was really those two.

5 Building

The product itself is submitted separately in the Product Artifact ZIP — per the assignment, this section only refers to it.

One connecting sentence (draft): the build followed the priority order from Section 4 — university discovery first, then a company arm on the same no-database principle, then consolidation into a single entry-point skill; the ZIP contains the skill package, install guide, and evaluation harness.

6 Testing

Assignment asks for: what was tested, who tested it, how, main feedback, and problems / surprises / confirmations — with test notes and quotes as evidence.

What we have

- Extensive **self-run evaluation** (this is real testing and should be presented as such, but it is not user testing):
  - Recall/precision measured per faculty against hand-curated ground truth, with the skill scoring far above a plain-Claude baseline.

  - A steering test showing that different student profiles genuinely produce different option maps (not one generic answer).
  - An independent, deliberately skeptical review that challenged our own "it works" verdict and produced a punch list — including catching that some of our numbers weren't as clean as claimed. Honest material, good for the portfolio.
  - A surprise worth telling: our first eval design was circular (the fixture setup let the system be graded on data it had effectively seen) — we caught it and moved to live evals. That's a genuine "problem discovered through testing."
- **Informal in-between tests** with people around the project happened, including a hands-on session with the course professor that produced direct feedback — but these were never written down properly.
- **Real student beta test: not yet run.** Protocol, capture sheet, and recruiting message are written and ready; nobody outside the project has used the tool independently yet. The portfolio must state this honestly.

Evidence to attach

- Evaluation summary and numbers: `docs/thesis-report/03-hardening-and-evaluation/README.md`, `findings/no_db_universal_skill/2026-07-03-eval-aggregate-scorecard.md`
- The one clean measurement on the current architecture: `findings/no_db_universal_skill/2026-08-08-theology-blind-run.md`
- Skeptical review: `findings/no_db_universal_skill/2026-07-05-fable-1.0-readiness-review.md`
- Beta-test protocol (ready, unexecuted): `findings/no_db_universal_skill/2026-08-08-beta-test-protocol.md`

So what / Now what

- Depends on the open items below — this section can't be finished until the informal tests are reconstructed and (ideally) the beta test has run.

Open — needs the team

- Reconstruct each informal test as a proper record: who tested (role is enough, no names needed), what they were asked to do, what they actually said — ideally one verbatim quote each — and what surprised us or confirmed our assumptions.
- The professor's hands-on session produced the single most important piece of feedback in the whole project (value = reflection, not proposals). Reconstruct that session concretely: what was demoed, what was said, what changed because of it.
- Decide: do we run the beta test before submission? If yes — results go here. If no — the portfolio says the protocol is ready, explains why it hasn't run, and Section 7 carries it as the next step. Either is defensible; silently omitting it is not.

7 Iteration and Next Steps

Assignment asks for: changes made from feedback, changes planned but not done, remaining limitations, distribution/shipping strategy, maintenance/handover strategy, and the possible role of AI in maintaining or evolving the project.

What we have

- Changes made based on feedback (pick the 2–3 clearest for the portfolio):
  - The pivot itself — the largest feedback-driven change in the project.
  - The professor's reframing ("the value is the student's reflection") was built into the product:

the skill now records the student's thesis self-understanding before and after the search.
  - Multiple concrete skill fixes that came directly out of eval findings (e.g. verification rules that were losing correct candidates).
- Changes planned but not implemented: the real student beta test; re-measuring the faculties whose numbers predate the architecture change; a designed-but-never-run experiment testing our central claim (that the skill's advantage comes from curated local scope and would erode as scope grows).
- Remaining limitations: only one clean measurement on the current architecture; company arm weaker than the university arm; coverage honesty is a feature but also a limit.
- Distribution/shipping (documented, real): the skill package is publicly released on GitHub with an install guide, and a cold install was tested. Distribution = "install these markdown folders," which is the whole point.
- Maintenance/handover: a survival plan exists — because the product is markdown-only (no database, no backend, no API keys), almost nothing can rot; main risks are the skill format changing, university sites restructuring, and rules going stale. Estimated cost: a couple of hours per semester. Ownership options are ranked, **but no owner has been named.**
- Role of AI in maintenance — we have an unusually strong answer: the repo ships an operating guide written *for future AI agents* (`AGENTS.md`), so the intended maintainer of this AI-built product is itself an AI agent, with a human owner spending ∼2 h/semester steering it. This maps exactly onto the assignment's question and should be featured, not buried.

Evidence to attach

- Open work and rejected ideas: `docs/thesis-report/04-open-work/README.md`
- Survival/maintenance plan: `findings/no_db_universal_skill/2026-08-09-survival-and-maintenance-plan.md`
- Public release + install guide: GitHub release page, `INSTALL.md`
- Agent maintainer guide: `AGENTS.md`

So what (draft)

- Candidate for the most important iteration learning: every major improvement came from an outside signal (interviews, professor feedback, the skeptical review) — never from us admiring our own output. To be confirmed/sharpened by the group.

Now what (draft)

- If the project continued, the next meaningful step is putting the tool in real students' hands (the beta test) — because it's the only claim in the whole journey we haven't tested yet.

Open — needs the team

- Ownership must be resolved for the portfolio: does one of us take the ∼2 h/semester maintenance role, does it go to someone at the university, or do we honestly write "unowned — and here is what that means for the product's lifespan"? Any of the three works; the gap can't stay silent.
- Has anything changed since the last documented project state (new runs, new feedback, new decisions)? The portfolio should reflect where we actually are at submission, not where the repo log stops.
- Confirm or replace the "most important iteration learning" draft above — this should be a sentence the whole group stands behind.

Submission checklist — current status

Checklist item	Status
First problem idea	✓ covered (Section 1)
Initial project idea / proposal	✓ covered — *needs one early artifact*
Relevant users / user groups	✓ covered — *needs persona + student-side account*
Initial and refined problem definitions	● content there — *needs the two one-sentence versions*
Proposed solution + value proposition	● content there — *needs the agreed one-sentence value prop*
High-level product description	✓ covered (Section 5 + ZIP)
User testing and feedback	✗ weakest point — *informal tests must be written up; beta decision needed*
Iteration / planned changes	✓ covered (Section 7)
Shipping / access / maintenance strategy	✓ covered — *except the unresolved owner*
Concrete evidence, not just claims	✓ strong — repo is full of citable artifacts
Clear and concise structure	pending — this draft becomes 4–6 pages once gaps close